

4.5 GBM(Gradient Boosting Machine)

목차

1. 부스팅과 GBM 개요

1. 부스팅(Boosting)이란
2. AdaBoost의 동작 원리
3. GBM — 경사 하강법 기반 부스팅
4. 배깅과 부스팅 비교

2. GBM으로 HAR 데이터 분류

1. 데이터 로딩과 학습
2. 수행 시간 측정 결과

3. GBM 하이퍼 파라미터

1. 주요 파라미터
2. GridSearchCV 예제

4. 사이킷런 1.x의 GBM 속도 문제와 대안

1. 측정된 학습 속도
2. HistGradientBoostingClassifier

5. 핵심 요약 및 머신러닝 활용 포인트

1. 부스팅과 GBM 개요

1.1 부스팅(Boosting)이란

부스팅 알고리즘은 여러 개의 약한 학습기(weak learner)를 **순차적으로 학습·예측**하면서, **잘못 예측한 데이터에 가중치를 부여**해 오류를 개선해 나가는 방식입니다.

4.4장의 배경이 **여러 트리를 동시에(병렬로) 만들어 다수결**을 했다면, 부스팅은 **한 번에 하나씩 순차적으로** 만들면서 **앞선 모델의 실수를 다음 모델이 보완**합니다.

■ 직관적인 비유

배경이 **여러 전문가에게 독립적으로 물어본 뒤 다수결**을 하는 것이라면, 부스팅은 **한 명이 시험을 본 뒤 틀린 문제를 다음 사람이 집중적으로 공부**하고, 또 그 사람이 틀린 문제를 그다음 사람이 공부하는 식의 **릴레이 학습**입니다.

1.2 AdaBoost의 동작 원리

AdaBoost(Adaptive Boosting)는 **오류 데이터에 가중치를 부여**하면서 부스팅을 수행하는 대표적인 알고리즘입니다.

단계	내용
Step 1	첫 번째 약한 학습기가 분류 기준을 만들어 데이터를 분류합니다. 일부는 오분류 됩니다.
Step 2	오분류된 데이터에 가중치를 부여 (크기를 키움)합니다. 다음 학습기가 이 데이터를 더 중요하게 취급하게 됩니다.
Step 3	두 번째 학습기가 가중치가 반영된 데이터로 새 분류 기준을 만듭니다. 또다시 오분류가 발생하면 가중치를 갱신 합니다.
Step 4	이 과정을 지정한 횟수만큼 반복한 뒤, 모든 약한 학습기의 결과를 가중 결합 해 최종 예측을 만듭니다.

개별 약한 학습기는 **단순한 규칙 하나**에 불과하지만, 이들을 결합하면 **훨씬 정교한 결정 경계**가 만들어집니다.

1.3 GBM — 경사 하강법 기반 부스팅

GBM(Gradient Boosting Machine)은 AdaBoost와 유사하지만, **가중치 업데이트를 경사 하강법(Gradient Descent)**으로 수행한다는 결정적 차이가 있습니다.

■ 경사 하강법이 사용되는 방식

오류값을 **실제값 - 예측값**이라고 할 때, GBM은 이 **오류(잔차, residual)**를 다음 트리가 예측하도록 학습시킵니다.

1. 첫 번째 트리가 예측을 수행 → 오차 발생
2. 두 번째 트리는 **원래 정답이 아니라 "첫 트리의 오차"**를 맞히도록 학습
3. 세 번째 트리는 **앞선 두 트리의 남은 오차**를 맞히도록 학습
4. ... 반복하며 **오차를 점진적으로 줄여** 나감

이렇게 **오류를 최소화**하는 방향성을 가지고 반복적으로 **가중치를 업데이트**하는 것이 경사 하강법이며, 이것이 GBM의 핵심입니다.

1.4 배경과 부스팅 비교

구분	배경 (랜덤 포레스트)	부스팅 (GBM)
----	--------------	-----------

학습 방식	병렬 — 트리들이 독립적	순차 — 앞 트리에 의존
목적	분산(variance) 감소	편향(bias) 감소
개별 트리	깊고 복잡함	얕고 단순함 (약한 학습기)
속도	빠름 (병렬화 가능)	느림 (순차라 병렬화 불가)
과적합	강함	상대적으로 취약 (규제 필요)
일반 성능	안정적	일반적으로 더 높음

2. GBM으로 HAR 데이터 분류

2.1 데이터 로딩과 학습

```
from sklearn.ensemble import GradientBoostingClassifier
import time
import warnings
warnings.filterwarnings('ignore')

X_train, X_test, y_train, y_test = get_human_dataset()

# GBM 수행 시간 측정을 위한. 시작 시간 설정.
start_time = time.time()

gb_clf = GradientBoostingClassifier(random_state=0)
gb_clf.fit(X_train, y_train)
gb_pred = gb_clf.predict(X_test)
gb_accuracy = accuracy_score(y_test, gb_pred)

print('GBM 정확도: {0:.4f}'.format(gb_accuracy))
print("GBM 수행 시간: {0:.1f} 초 ".format(time.time() - start_time))
```

줄	코드	상세 설명
1	from sklearn.ensemble import GradientBoostingClassifier	GBM 분류기를 임포트합니다. 랜덤 포레스트와 같은 ensemble 모듈에 있습니다.
2	import time	수행 시간을 측정 하기 위한 표준 라이브러리입니다. GBM 실습에서 시간 측정이 중요한 이유는 속도가 GBM의 핵심 약점 이기 때문입니다.
6	X_train, X_test, y_train, y_test = get_human_dataset()	4.2·4.4장과 동일한 HAR 데이터 를 로딩합니다. 학습 7,352건 × 561피처, 테스트 2,947건입니다.
9	start_time = time.time()	현재 시각을 초 단위 실수 로 저장합니다. 나중에 차이를 구해 경과 시간을 계산합니다.
11	gb_clf = GradientBoostingClassifier(random_state=0)	모든 하이퍼 파라미터를 기본값 으로 생성합니다. 기본 n_estimators=100 , learning_rate=0.1 , max_depth=3 입니다.
12	gb_clf.fit(X_train, y_train)	학습을 수행합니다. 여기서 대부분의 시간이 소요 됩니다.
17	time.time() - start_time	종료 시각에서 시작 시각을 빼 총 경과 시간 을 구합니다.

■ GBM은 왜 이렇게 느린가

HAR 데이터는 **6개 클래스**의 다중 분류입니다. 사이킷런의 GBM은 다중 분류에서 **각 반복(stage)마다 클래스 수만큼 트리**를 만듭니다. 따라서 `n_estimators=100` 이면 실제로는 $100 \times 6 = 600$ 개의 트리가 순차적으로 만들어집니다.

게다가 부스팅은 **앞 트리의 결과에 의존**하므로 **병렬 처리가 원천적으로 불가능**합니다. 랜덤 포레스트가 `n_jobs=-1`로 속도를 크게 높일 수 있었던 것과 대조적입니다.

2.2 수행 시간 측정 결과

△ 본 문서 작성 환경에서의 실행 결과 안내

본 강의자료를 작성한 실행 환경은 **CPU 2코어 / 메모리 3GB**로, 개별 실행이 **45초 이내**에 완료되어야 하는 제약이 있었습니다.

기본 설정 GBM(100 stage)의 학습 속도를 **실측**한 결과는 다음과 같습니다.

▶ 실측값 (본 문서 작성 환경)
 5 stage 학습 시간 : 35.8초
 1 stage당 평균 : 약 7.2초
 100 stage 전체 예상 소요 시간 : 약 716초 (약 12분)

이 환경에서는 **100 stage 전체 학습을 완료할 수 없어**, 아래 정확도 수치는 **교재 및 일반적인 실행 환경에서 보고되는 참고값**입니다.

GBM 정확도: 0.9386
 GBM 수행 시간: 약 700~900 초 (환경에 따라 상이)

대신 **동일 데이터에서 실제로 완주한 부스팅** 결과는 4절의 `HistGradientBoostingClassifier`를 참고하세요.

■ 다른 알고리즘과의 비교 (동일 HAR 데이터)

알고리즘	정확도	학습 시간	측정 여부
결정 트리 (튜닝)	0.8717	약 5초	실측
랜덤 포레스트 (기본)	0.9223	5.6초	실측
GBM (기본)	0.9386	약 12분	정확도는 참고값 시간은 실측 기반 추정
HistGradientBoosting	0.9325	33.2초	실측

GBM은 **랜덤 포레스트보다 정확도가 1.6%p 높지만 학습 시간은 100배 이상 걸립니다**. 이 성능과 시간의 **트레이드오프**가 GBM의 본질적인 특징이며, 이를 해결하기 위해 나온 것이 **XGBoost(4.6)**와 **LightGBM(4.7)**입니다.

3. GBM 하이퍼 파라미터

3.1 주요 파라미터

파라미터	설명	기본값
<code>n_estimators</code>	생성할 약한 학습기(부스팅 단계) 의 개수. 많을수록 성능이 좋아질 수 있으나 수행 시간이 비례해 증가 합니다.	100

learning_rate	각 학습기가 결과에 반영되는 비율(0~1) . 작을수록 신중하게 학습해 성능이 좋아지지만 더 많은 트리가 필요합니다. 크면 빠르지만 최적점을 지나칠 수 있습니다.	0.1
max_depth	개별 트리의 최대 깊이. 부스팅은 약한 학습기 를 쓰므로 얕게(3~8) 유지합니다.	3
subsample	각 트리가 학습에 사용할 데이터 샘플링 비율 . 1이면 전체 사용. 0.5~0.8로 낮추면 과적합을 억제 할 수 있습니다.	1.0
loss	경사 하강법에 사용할 손실 함수	'log_loss'

■ n_estimators와 learning_rate의 관계

두 파라미터는 **서로 보완적**입니다. **learning_rate**를 **작게** 하면 각 트리의 기여가 줄어들므로 **n_estimators**를 **크게** 해야 합니다. 일반적으로 **학습률을 낮추고 트리를 늘리는 쪽**이 성능이 좋지만 시간이 오래 걸립니다.

실무 전략은 **학습률을 0.05~0.1로 고정**하고, **조기 중단으로 적절한 트리 개수를 자동으로 찾는 것**입니다(4.6장 XGBoost에서 실습).

3.2 GridSearchCV 예제

아래는 책에서 설명드리지는 않지만 GridSearchCV로 GBM의 하이퍼 파라미터 튜닝을 수행하는 예제 입니다.
 ### 사이킷런이 1.X로 업그레이드 되면서 GBM의 학습 속도가 현저하게 저하되는 문제가 오히려 발생합니다.
 ### 아래는 수행 시간이 오래 걸리므로 참고용으로만 사용하시면 좋을 것 같습니다.

```
from sklearn.model_selection import GridSearchCV

params = {
    'n_estimators':[100, 500],
    'learning_rate' : [ 0.05, 0.1]
}
grid_cv = GridSearchCV(gb_clf , param_grid=params , cv=2 ,verbose=1)
grid_cv.fit(X_train , y_train)
print('최적 하이퍼 파라미터:\n', grid_cv.best_params_)
print('최고 예측 정확도: {0:.4f}'.format(grid_cv.best_score_))
```

줄	코드	상세 설명
1~3	### 주석	교재 저자가 직접 남긴 경고입니다. 사이킷런 1.x에서 GBM 학습 속도가 크게 저하되었음 을 명시하고 있습니다.
7~10	params = {'n_estimators':[100,500], 'learning_rate':[0.05,0.1]}	조합은 $2 \times 2 = 4$ 가지입니다.
11	GridSearchCV(gb_clf, param_grid=params, cv=2, verbose=1)	cv=2 로 최소한의 폴드만 사용해도 $4 \times 2 = 8$ 번의 GBM 학습이 필요합니다.

⚠ 이 코드의 실제 소요 시간

본 문서 환경의 실측 기준(1 stage $\approx$ 7.2초)으로 계산하면 다음과 같습니다.

```
n_estimators=100 학습 2회 :  $100 \times 7.2 \times 2 \approx 1,440$ 초
n_estimators=500 학습 2회 :  $500 \times 7.2 \times 2 \approx 7,200$ 초
learning_rate 2가지이므로  $\times 2$ 
```

합계 : 약 17,280초 $\approx$ 4.8시간

따라서 이 셀은 실무에서 그대로 실행하기 어렵습니다. 교재 저자가 "참고용으로만"이라고 한 이유입니다. GBM 계열을 튜닝해야 한다면 **XGBoost나 LightGBM으로 대체**하는 것이 현실적입니다.

```
# GridSearchCV를 이용하여 최적으로 학습된 estimator로 predict 수행.
gb_pred = grid_cv.best_estimator_.predict(X_test)
gb_accuracy = accuracy_score(y_test, gb_pred)
print('GBM 정확도: {0:.4f}'.format(gb_accuracy))
```

줄	코드	상세 설명
2	grid_cv.best_estimator_.predict(X_test)	<code>refit=True</code> 기본값 덕분에 최적 파라미터로 재학습된 모델 이 준비되어 있습니다. 별도의 <code>fit()</code> 없이 바로 예측합니다.

4. 사이킷런 1.x의 GBM 속도 문제와 대안

4.1 측정된 학습 속도

본 문서에서는 GBM의 속도 문제를 **추측이 아니라 실측**으로 확인했습니다.

```
import time
from sklearn.ensemble import GradientBoostingClassifier

t = time.time()
GradientBoostingClassifier(random_state=0, n_estimators=5).fit(X_train, y_train)
print('5 stages: %.1fs -> 100 stages 추정 %.0f초' % (time.time()-t, (time.time()-t)*20))
```

▶ 실행 결과

5 stages: 35.8s -> 100 stages 추정 716초 (11.9분)

■ 이 측정이 알려 주는 것

- 1 stage당 **약 7.2초**. HAR은 6클래스이므로 stage마다 트리 6개를 만듭니다 → 트리 하나당 **약 1.2초**.
- 피처가 **561개**로 많아 각 분할에서 검토할 후보가 매우 많습니다. GBM은 **모든 피처를 전수 검토**하므로(`max_features=None`이 기본) 이 비용이 그대로 반영됩니다.
- 랜덤 포레스트는 `max_features='sqrt'`가 기본이라 $\sqrt{561} \approx 24$ 개만 검토합니다. 여기서 **약 23배의 속도 차이**가 발생합니다.

4.2 HistGradientBoostingClassifier

사이킷런은 GBM의 속도 문제를 해결하기 위해 **HistGradientBoostingClassifier**를 제공합니다. LightGBM에서 영감을 받은 **히스토그램 기반** 구현으로, 연속값을 **256개 구간(bin)**으로 이산화해 분할 후보를 대폭 줄입니다.

```
import time
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import accuracy_score

t = time.time()
hgb_clf = HistGradientBoostingClassifier(random_state=0)
hgb_clf.fit(X_train, y_train)
elapsed = time.time() - t

pred = hgb_clf.predict(X_test)
print('HistGradientBoosting 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
print('HistGradientBoosting 수행 시간: {0:.1f} 초'.format(elapsed))
```

줄	코드	상세 설명
2	from sklearn.ensemble import HistGradientBoostingClassifier	사이킷런 0.21부터 제공되는 고속 GBM입니다. 별도 패키지 설치 없이 사용할 수 있습니다.
6	HistGradientBoostingClassifier(random_state=0)	기본값으로 생성합니다. max_iter=100 (GBM의 n_estimators에 해당), learning_rate=0.1 입니다.
7	hgb_clf.fit(X_train, y_train)	사용법은 GBM과 완전히 동일 합니다. 클래스 이름만 바꾸면 됩니다.

▶ 실행 결과 (실측)

```
HistGradientBoosting 정확도: 0.9325
HistGradientBoosting 수행 시간: 33.2 초
```

■ 결과 해석 — 21배 빠른 부스팅

구분	GradientBoosting	HistGradientBoosting
정확도	0.9386 (참고값)	0.9325 (실측)
학습 시간	약 716초 (추정)	33.2초 (실측)
속도 비교	기준	약 21배 빠름
분할 방식	모든 값을 전수 검토	256개 bin으로 이산화
병렬 처리	불가	가능 (OpenMP)

정확도는 **0.6%p** 낮지만 학습 시간은 **21분의 1**입니다. 이산화 과정에서 미세한 정보 손실이 있지만, **실용적으로는 압도적으로 유리한 교환**입니다.

이 결과는 4.7장에서 배울 **LightGBM의 히스토그램 기반 접근**이 왜 그렇게 강력한지를 사이킷런 안에서 미리 보여 주는 사례이기도 합니다.

5. 핵심 요약 및 머신러닝 활용 포인트

5.1 핵심 개념 요약

개념	핵심 내용
부스팅	약한 학습기를 순차적으로 학습하며 이전 오류를 보정
AdaBoost	오분류 데이터에 가중치를 부여하는 방식
GBM	가중치 업데이트를 경사 하강법으로 수행. 잔차를 학습
배깅 vs 부스팅	배깅=병렬·분산 감소 / 부스팅=순차·편향 감소

5.2 핵심 API 요약표

파라미터	역할	기본값	튜닝 방향
<code>n_estimators</code>	부스팅 단계 수	100	늘리면 성능 ↑ 시간 ↑
<code>learning_rate</code>	학습률	0.1	낮추면 성능 ↑, 트리 수 증가 필요
<code>max_depth</code>	개별 트리 깊이	3	부스팅은 얇게 유지
<code>subsample</code>	데이터 샘플링 비율	1.0	과적합 시 0.5~0.8
대안 클래스			
<code>HistGradientBoostingClassifier</code>	히스토그램 기반 고속 GBM	<code>max_iter=100</code>	대용량 데이터에 강력 권장

5.3 알고리즘 성능 비교 (동일 HAR 데이터)

알고리즘	계열	정확도	학습 시간
결정 트리 (튜닝)	단일 트리	0.8717	약 5초
랜덤 포레스트 (기본)	배깅	0.9223	5.6초
HistGradientBoosting	부스팅(고속)	0.9325	33.2초
GBM (기본)	부스팅	0.9386	약 12분

정확도는 단일 트리 < 배깅 < 부스팅 순으로 향상되지만, 학습 시간은 정반대 방향으로 급증합니다.

5.4 머신러닝 활용 포인트

- **GBM은 성능은 좋지만 느리다** : 이 트레이드오프를 이해하는 것이 4장의 핵심입니다. 정형 데이터에서 부스팅은 가장 강력한 계열이지만, 순차 학습이라는 구조적 제약이 있습니다.
- **사이킷런 GBM은 실무에서 잘 쓰지 않는다** : 개념 학습용으로는 훌륭하지만, 실제 프로젝트에서는 **XGBoost, LightGBM, HistGradientBoosting** 중 하나를 사용합니다.
- **learning_rate와 n_estimators를 함께 조정** : 둘은 독립적이지 않습니다. 학습률을 낮추면 트리를 늘려야 합니다.
- **얇은 트리를 많이 쌓는다** : 부스팅의 `max_depth` 기본값이 3인 이유입니다. 배깅과 정반대의 철학입니다.
- **시간 측정을 습관화** : 본 장치럼 `time.time()` 으로 학습 시간을 기록해 두면 **알고리즘 선택의 근거**가 됩니다.
- **다음 단계 : XGBoost(4.6)**는 GBM에 병렬 처리·규제·조기 중단을 더해 속도와 성능을 모두 개선했고, **LightGBM(4.7)**은 여기서 한 단계 더 나아가 **리프 중심 분할**로 속도를 극대화했습니다.

⚠ 안정화 버전 참고 (scikit-learn 1.7.x 기준)

- **속도 저하 문제** : 교재 저자가 지적한 대로 사이킷런 1.x에서 `GradientBoostingClassifier` 의 학습 속도가 이전 버전 대비 크게 느려졌습니다. 본 문서에서 실측한 **1 stage당 7.2초**가 그 근거입니다.
- `loss` 파라미터의 기본값 이름이 `'deviance'` → `'log_loss'` 로 변경되었습니다(1.1부터, 1.3에서 구 이름 제거).
- `criterion='mse'` 는 제거되었고 `'friedman_mse'` (기본) 또는 `'squared_error'` 를 사용합니다.
- **권장**: 데이터가 **1만 건 이상**이면 `GradientBoostingClassifier` 대신 `HistGradientBoostingClassifier` 를 사용하세요. 사이킷런 공식 문서도 이를 권장합니다.
- `fit( )` 에 2차원 `y` 를 넘기면 `DataConversionWarning` 이 발생합니다. `y_train.values.ravel()` 로 변환하세요.